

A machine learning approach to circumventing the curse of dimensionality in discontinuous time series machine data

Oluseun Omotola Aremu^{*,a}, David Hyland-Wood^b, Peter Ross McAree^a

^a School of Mechanical and Mining Engineering, The University of Queensland, QLD, 4072, Australia

^b School of Information Technology and Electrical Engineering, The University of Queensland, QLD, 4072, Australia

ARTICLE INFO

Keywords:

Machine learning
Deep learning
Dimension reduction
Manifold learning
Predictive maintenance
Prognostics
Partial differential equations

ABSTRACT

The growing interest in artificial intelligence has led to current data-driven predictive maintenance (PdM) relying on machine learning (ML) algorithms. Although ML algorithms are useful for data-intensive analysis, research shows that their performance and reliability are reduced when high-dimensional data is used for training and testing. Raw machine data can be high-dimensional due to multi-sensor measurements and discontinuous due to the wide ranges of parameter variations during continuous sensor measurements. While standard dimension reduction methods such as principal component analysis are often applied to circumvent high-dimensionality, they are often unreliable when the data is discontinuous. This paper presents a ML-based dimension reduction framework to circumvent the challenges of high-dimensional discontinuous machine data. This framework minimizes discontinuity by clustering observations based on the dataset's modality. The modality is identified using a kernel density estimation parameterized using a heat diffusion solution to the approximate mean integrated squared error. Then, low-dimension representations of each cluster are learned using Laplacian eigenmaps embedding. Finally, the original time sequence of observations across the low-dimensional clusters is used to re-index the observations into a continuous low-dimension feature set. We demonstrate the framework's utility on common ML-based PdM analysis using the Commercial Modular Aero-Propulsion System Simulation dataset.

1. Introduction

Predictive maintenance (PdM) is the monitoring of a machine or system's condition over its life cycle to provide a prognosis to when maintenance is required [21]. In this domain, complex systems are monitored through multiple sensing instruments [1,21]. This often translates to high-dimensional multivariate time series data [21]. Also, it is common that the monitored systems experience a wide range of operational use and environments over their life cycle [1,24]. In terms of data characteristics, wide ranges of variations translate to a multi-modal dataset, and such modality variations in a dataset translate to discontinuity in its distribution [31,34]. Given the characteristics of machines, a dataset generated in the PdM domain is expected to consist of samples that generate high-dimensional discontinuous datasets.

Along with the monitoring of machines, PdM entails data-driven analytics in predicting machine failure, detecting faults within a machine, and detecting when a machine is operating out of the bounds of its normal operating condition [24]. As data-driving technology continues to mature, PdM tools are increasingly dependent on machine

learning (ML) based artificial intelligence technology [1,21,25].

Although ML algorithms have proven to be state-of-the-art in improving PdM capabilities, they are not perfect. Their reliability can be reduced by the data form used to train and test algorithms [21]. These limitations can result in ineffective and unreliable PdM tools due to increased computation complexity, time, and reduced accuracy [21].

Specifically, high-dimensional (multivariate) data highlight the limitations of ML algorithms [10]. These variations in data are common and make it difficult for ML algorithms to contrive information from observed data. The inability of ML algorithms to manage high-dimensional data is known as the *curse of dimensionality* [37].

The problem of high-dimensional data is of concern to the PdM domain; specifically, discontinuous high-dimensional data [15,25,26,31]. However, standard dimension reduction methods are not enough to address the *curse of dimensionality* problem in machine data when discontinuity is present. This gap in innovation could leave ML-based PdM algorithms unreliable and susceptible to inaccurate analysis.

This paper presents a machine learning dimension reduction framework (MLDR-framework) that allows high-dimensional discontinuous

* Correspondence author.

E-mail addresses: o.aremu@uq.edu.au (O.O. Aremu), d.hylandwood@uq.edu.au (D. Hyland-Wood), p.mcaree@uq.edu.au (P.R. McAree).

<https://doi.org/10.1016/j.ress.2019.106706>

Received 8 April 2019; Received in revised form 8 October 2019; Accepted 20 October 2019

Available online 09 November 2019

0951-8320/ © 2019 Elsevier Ltd. All rights reserved.

machine data to be mapped to a continuous low-dimension that: (1) retains the highest possible measure of information contained in the original data, (2) improves utility compared to original data, and (3) ensures a continuous lower-dimension representation invariant to influences of time-varying operating conditions. Overall, the MLDR-framework is a unique data-driven framework capable of achieving a lower-embedding of nonlinear data containing samples with high parameter variation (i.e discontinuous data).

1.1. Curse of dimensionality

The *curse of dimensionality* defines a fixed condition of ML algorithms in which the higher the dimension space, the more dense data samples required [23]. For a ML algorithm, a 100 by 200 point cloud in which each point could be a variable, can lead to a dimensionality of 20,000 features which will lead to an observed sparsity of data. This sparsity inhibits ML algorithms from deriving strong statistical inferences based on data point relations, which is the core principle of most ML algorithms [23].

Feature selection [11,16,30,51], and *feature projection* [26,34] are commonly applied to circumvent the *curse of dimensionality* problem. As there exist many methods within each category, for brevity, this work is restricted to the discussion of *feature projection* through dimension reduction.

1.2. Organization

This paper is organized as follows: Section 4 provides a use-case description of discontinuous data in machine data, Section 2 provides a literature review on dimension reduction and ML in PdM, while Section 3 discussed the use of deep learning in ML-based PdM. Section 5 introduces the MLDR-framework, and Section 6 explains the experiment set-up used to evaluate the MLDR-framework's performance. The results of the experiments are described in Section 7, followed by a discussion and conclusion in Section 8.

2. Literature review

2.0.1. Linear methods

Principal component analysis (PCA), independent component analysis (ICA) and multidimensional scaling (MDS), are some of the more widely used linear reduction methods. Like other linear methods, the following are solely applicable when the features of the dataset are assumed to be linearly related.

PCA achieves a lower dimension representation by finding the linear combination of the original features through an eigenvalue solution to the covariance matrix generated by the original data [47]. The set of eigenvectors with the higher eigenvalues provide a set of column vectors that are the lower dimension space representation of the original data.

Although MDS and PCA generate similar results, MDS achieves a low-dimensional representation by preserving the euclidean pairwise distance between data points in their high-dimensional space [47]. As PCA provides a solution using the original dataset's covariance matrix, MDS uses the original distance or affinity matrix of the high-dimensional data points.

ICA's mapping to a lower dimension depends on the correlation matrix generated by the original dataset [47]. By using a factor analysis solution and searching for rotations that lead to adjacent components, it generates a set of independent column vectors of a low-dimensional space.

The assumption of linearity in the observed data by these and other linear methods is not always true for real-world data [47]. Therefore, they are often limited in their applications due to their inaccurate low-dimensional representations when applied to nonlinear data.

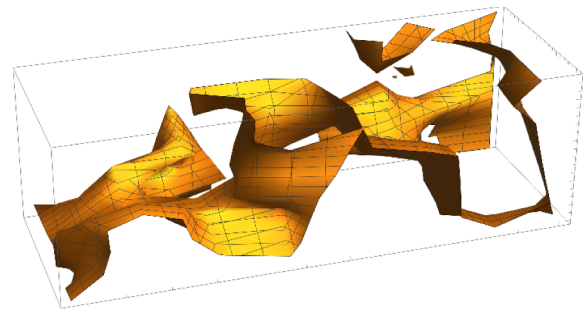


Fig. 1. Example of a discontinuous manifold with multiple discontinuities.

2.0.2. Nonlinear methods

Nonlinear or manifold learning methods assume that all observed data lie on a differentiable manifold [27,36,49]. Per the mathematical definition of a manifold [14], for a collection of data points on a surface to be considered a differentiable manifold, the set of data points must be on a continuous (regular) and smooth (differentiable) surface [14,27]. As manifold learning assumptions are based on the mathematical definition of a differentiable manifold, associated algorithms are limited to datasets in which the observed points originate from a continuous manifold.

Manifold learning algorithms assume data is generated due to a continuously varying set of parameters, and also expect the parameter variation to be limited. Therefore, one may assume that all observed data originate from the same distribution [27]. This is not common of data generated from real-world systems for reasons we will demonstrate later in this work. Both a discontinuous and continuous manifold are illustrated in Fig. 1 and Fig. 2 respectively.

In the event data samples contain a large range of parameters, one can expect discontinuities between the data samples [27]. This would result in a discontinuous manifold as shown in Fig. 1, and manifold learning methods would be limited in their ability to generate a faithful low-dimensional representation as explained in [14,27].

2.0.3. Manifold learning methods

As many manifold learning methods exist, for brevity, this work will focus only on common methods that have been widely used: locally linear embedding (LLE) [47,49], isometric mapping (Isomap) [47], and Laplacian eigenmaps (LEM) [6,47]. As these are the commonly used modern methods, their representation are compared with that of the MLDR-framework.

LLE maps data points to a lower dimension manifold where the linear distance relationship between neighboring points in the higher dimension space is preserved [49]. An eigenvalue problem is solved using a cost function that minimizes the high-to-low dimension

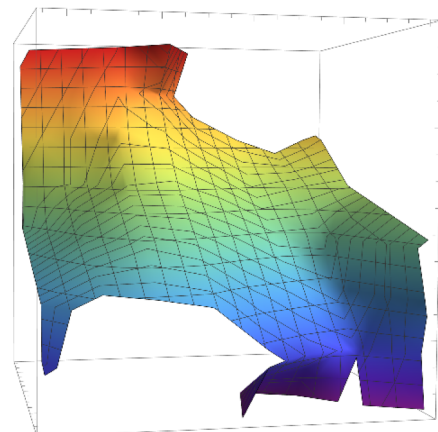


Fig. 2. Example of a continuous manifold.

embedding difference. Thus, providing eigenvectors and eigenvalues that describe the lower-dimension data. Given a high dimensional dataset $X \in R^{n \times p}$ with n features and p observations:

LLE identifies the neighbors of each data point x_i by finding the k nearest neighbors through a euclidean distance measure resulting in a row vector of weights W in which

$$\sum_{j \in p} w_{ij} = 1. \quad (1)$$

The weights that best describe the reconstruction of x_i per its neighbor relationship is determined by the cost function

$$\min_W \sum_{i \in n} \|X_i - \sum_{j \in p} w_{ij} X_j\|^2. \quad (2)$$

This minimization provides a mapping function defined by the weights and describes the intrinsic geometric relationships between all points per their repetitive neighborhoods.

Once the weights have been determined, X is then mapped to a lower dimension subspace Y

$$Y = \Phi(X), \quad (3)$$

by finding the ideal low-dimension subspace embedding vector Y_i through minimizing the function:

$$\min_Y \sum_{i \in n} \|Y_i - \sum_{j \in p} w_{ij} Y_j\|^2, \quad (4)$$

which describes the embedding cost function.

In a simpler form, as described in [38], LLE can be simplified to

$$\min_Y \text{Tr}(Y^T Y L), \quad (5)$$

where

$$L = (I - W)^T (I - W).$$

This form requires the embedding vector to be of a unit covariance $Y^T Y = I$. The embedding cost function is then a solution where the columns of Y^T are the eigenvectors of the lowest eigenvalues of L , and thus identifying the lower embedding vectors.

Isomap, an extension of MDS, maps high-dimensional data points to a lower dimension subspace by evaluating the distance relationship between the high-dimensional points based on their geodesic distance [47]. The geodesic distance represents the shortest path along the curved surface of the smooth manifold measured as if the surface were flat [27]. Given a high dimensional dataset $X \in R^{n \times p}$ with n features and p observations:

Isomap creates a kernel distance matrix G based on the matrix of inner products of $X^T X$ where

$$X^T X = -\frac{1}{2} H G^{(X)} H. \quad (6)$$

In the standard form of Isomap, $G := (g_{ij})$ is the geodesic distance matrix determined by Dijkstra's algorithm [44], and $H = I - \frac{1}{p} e e^T$ is the centering matrix.

Using the defined geodesic distance matrix, the embedding function is the solution of

$$\min_Y \sum_{i \in n} \sum_{j \in p} (g_i^T g_j - y_i^T y_j)^2, \quad (7)$$

where

$$Y = \lambda^{\frac{1}{2}} V^T.$$

Both V and λ denote eigenvectors and eigenvalues respectively. The eigenvectors $V e R^d$ in Y , where ($d < n$), are the lower dimension representations of the original data.

LEM maps to a lower dimension subspace by applying the Laplace-Beltrami [6] operator function to a kernel weighted embedding map or graph, and minimizing the map's squared gradient. In the graph, each node is a data point in the observed dataset, and the edges are weighted by the kernel [6]. Given a high dimensional dataset $X \in R^{n \times p}$ with n features and p observations:

LEM creates a weighted graph G , of X nodes connected by neighboring points defined by k nearest neighbors (free parameter chosen a priori). The weighting W of the edges or connections are describe by

$$W_{ij} = e^{-\frac{\|X_i - X_j\|^2}{2k^2}} \quad (8)$$

for connected nodes, and $W_{ij} = 0$ otherwise.

$$V_{ii} = \sum_{j \in p} W_{ij}, \quad (9)$$

where V is a diagonal matrix.

Using the weighted graph, and provided the embedding vector to be of a unit covariance $Y^T Y = I$, an embedding map is obtained by

$$\min_Y \text{Tr}(Y^T L Y), \quad (10)$$

where the Laplacian function $L = V - W$. By solving the eigenvalue problem where

$$L Y = \lambda V Y, \quad (11)$$

The solutions of Y as the eigenvectors of L provide the lower dimension subspace of the original data points $Y e R^d$ where $d < n$.

Although all methods have been proven to be successful, they are often restricted to text, digit, and image datasets [36]. However, they are adaptable to time series data.

2.1. Dimension reduction in the predictive maintenance domain

The common requirement of PdM in industries, is to monitor a system's life cycle over varying operations and/or environments, and provide up-to-date probability of failure to provide information capable of influencing or guiding decision making [21,24]. In this work, a machine's range of operation describes its range of usage patterns (work load, flying altitude, operating speed, etc.) and its use in varying environmental conditions (ambient temperature, pressure, location, etc.).

Overall, PdM is applied to improve the reliability of systems during operation. In manufacturing, PdM can be used to predict events such as component outages in an assembly process [50]. Within the utilities domain, it can be used to predict outages in power equipment such as wind turbines [46]. In other industries such as manufacturing, it can be used to achieve optimal maintenance scheduling of high value machines [29].

Within all domains where PdM is applied, the ability to measure or identify faults and their implications are essential for reliable PdM analysis [21]. Quantifying and retaining the effects of faults in machine data during data manipulation is necessary to ensure low-dimensional representations remain conducive to PdM analytics.

Therefore, dimension reduction methods within PdM must retain information pertaining to faults and their implications on the observed machine or system. Through experiments in this work, we demonstrate how the MLDR-framework can be a robust solution for the domain of PdM.

3. Deep learning as an alternative to dimension reduction for machine data

The current state-of-the-art methods in ML have progressed to deep learning where inference is based on the use of deep artificial neural

network (DNN) architectures [21]. A DNN is the expansion of the single layer artificial neural network (ANN) to a multi-layer architecture in which there are multiple neuron layers between the input and output layer of the perceptron [23,40]. Also, a DNN can be the combination or addition of multiple ANNs [23,40]. For example, a DNN algorithm may consist of a convolutional neural network combined with a multi-layer perceptron for classification or regression tasks [23]. Common DNN methods also include model-based reinforcement learning [17].

Generally, DNNs were developed to generate better performance on ML-based tasks where the dataset is of very high-dimensionality, and the parameters of analysis are intractable for standard ML algorithms (e.g. support vector machine, decision tree, logistic regression) [23,40]. The success of DNNs in other fields (i.e. computer vision, robotics, finance, medicine) have motivated the current trend in PdM, where ML-driven PdM methods are based on DNNs [21,24].

In ML-driven PdM, DNNs are widely applied in problems where standard ML algorithms are capable of comparable or similar performance. This is generally due to an attempt to circumvent the need for dimension reduction or feature engineering of machine data [15,20,21,25,29,42,46]. Although DNNs have been theoretically applied in ML-driven PdM, DNNs introduce new challenges, and do not fully negate the need for dimension reduction or feature engineering of machine data as expected [5,20,25,32].

3.1. The curse of dimensionality in DNNs

Generally, DNNs were developed to circumvent the high dimensional characteristics of complex data [23,40]. They improve performance by learning the high-dimensional data's representations and extracting features to an abstract lower dimension space, which is learned through optimization aimed to improve the DNN's performance [23]. Although DNNs have been successfully applied to various data types, they remain susceptible to various forms of the *curse of dimensionality* as they generally require input data to be of very high dimension [12,32].

Another form of the *curse of dimensionality* is observed when the number of features p exceeds the number of observations n ; $p > n$. Naturally, to circumvent the *curse of dimensionality* one must increase n or reduce p . Given DNNs require a large p , a large n is required to circumvent the *curse of dimensionality* in DNNs. This often translates to the requirement of large volumes of data observations that are representative examples for the desired ML task [12,32]. As defined in various research that apply DNNs, the ability to meet this requirement of "enough data" is difficult, as there is often not enough representative data available in real-world applications [12,25,32].

Within the reliability engineering domain, preventive repair strategies are often applied to machines and they are rarely allowed to operate until failure. Therefore, failure is rarely observed and there are limited datasets with enough examples of failure data [21,24]. This scarcity of example failure data is prohibitive to reliable deep learning, as ML research show that if DNN algorithms do not observe an ideal number of observations n compared to the number of feature p , they perform poorly [12,25,32].

Therefore, in some scenarios, prior dimension reduction methods are necessary when relatively few example observations are available. ML-driven PdM tools based on DNNs that have not learned from an adequate number of observations may produce unreliable results. Therefore, the use of such tools may further lead to unreliable PdM solutions, and diminish the return of investment on machine reliability initiatives.

3.2. Hyperparameter optimization and feature selection in DNNs

As discussed in Russell and Norvig [37], the more parameters used by an algorithm to learn a function, the less generalizable the algorithm [37]. This implies that the algorithm is incapable of making prediction

on new data observations which were not observed during a learning process. DNNs generally require a high number of parameter optimization. Also, the number of parameters increase as the number of features increase (i.e. if p increases, the number of required parameters increases).

Regularization methods are often applied to increase the generalizability of DNNs [22,37]. Generally, regularization methods are applied to control the significance of each parameter [33]. In DNNs, regularization methods are applied to control the significance of each parameter of a neuron layer and neurons which serve as coefficients to the input data's features [23,40]. The commonly applied methods are L_1 regularization and L_2 regularization [33]. The L_1 regularization applies penalty terms which force values of parameters to be less influential or approximately zero during optimization to improve performance. The L_2 regularization applies penalty terms which force values of parameters to equally distributed weighting or equal significance during optimization to improve performance [22,33].

Per the definition of *feature selection* found in Blum et al. [7] and Guyon et al. [16], DNNs are performing *wrapper* based feature selection when applying L_1 or L_2 regularization during hyperparameter optimization to improve algorithm performance [33]. Furthermore, as regularization tuning in a DNN requires user input and interpretation, this process of feature selection can be considered arbitrary and case specific to the practitioners experience level. Also, regularization in a DNN requires user input [22,33]. Such inputs are generally not specific to data, but are instead specific to improving the DNN's performance, and may not transfer to proper interpretation of the applied data [22,33]. Feature selection through regularization in DNN is appropriate for general data and is not ideal for machine data, as machine data features contain properties that can be misinterpreted when applying feature selection criterion specific to general data characteristics [3].

3.3. The application of DNNs in predictive maintenance

This work does not claim that DNNs are not useful. In fact, the development of DNNs and their applications in PdM are exciting and should be further explored and developed. Alternatively, this work promotes the use of DNNs in PdM when applied with prior data manipulation methods specific to machine data. Also, due to the limitations and unpredictability of deep learning, there are potential challenges in the full adoption of DNNs in industries such as asset management, where civilian safety is of high concern.

Overall, there is a need to understand the characteristics of data applied to ML algorithms to progress the developments of ML-driven PdM. This work believes an effective adoption of ML in PdM remains at the level of standard ML algorithms (non-DNN), combined with dimension reduction methods ideal for machine data. The succeeding section details a ML-based dimension reduction framework dedicated to achieving a more useful low-dimensional representation of machine data.

4. Discontinuity in the C-MAPSS dataset

In the reliability engineering and PdM domain, machines are used in a range of varying operations during continuous sensor based condition monitoring. Due to the change of operating conditions, a machine can produce a varying range of sensor measurements corresponding to the operating parameters. In the case of a bearing system, a normal condition may produce different vibration measurement ranges depending on the operating condition's rotation speed parameter. Therefore, during the continuous monitoring of the bearing's life cycle, collected data is composed of measurements from multiple operating conditions. A dataset representation of such as system with sequenced observations and wide ranges of measurements is expected to contain discontinuity.

In this work, we explore the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) datasets (FD001, FD002, FD003, FD004)

to demonstrate the affects of varying operating conditions and how they contribute to a discontinuous dataset representation of a machine's life cycle. Each C-MAPSS dataset contains the time series run-to-failure life cycles of a fleet of different aircraft turbofan engines obtained from the NASA Prognostics Center of Excellence [39]. The C-MAPSS datasets are described as follows:

FD001: A fleet of turbofan engines that have experienced one (1) operating condition, and a degradation to failure due to one failure mode.

FD002: A fleet of turbofan engines that have experienced six (6) varying operating conditions, and a degradation to failure due to one failure mode.

FD003: A fleet of turbofan engines that have experienced one (1) operating condition, and experienced a degradation to failure due to two failure modes.

FD004: A fleet of turbofan engines that have experienced six (6) varying operating conditions, and experienced a degradation to failure due to two failure modes.

Fig. 3 provides a comparison in dataset characteristics between fleets FD001 and FD003 (experienced one operating condition), and fleets FD002 and FD004 (experienced six (6) operating conditions). As observed in Fig. 3b and 3d, the fleets with multiple operating conditions contain discontinuous sensor measurements with wide ranges of parameter variations and multi-modal distributions. While Fig. 3a and 3c show that fleets with no variations in operating conditions generated continuous sensor measurements and a single mode distribution. Furthermore, in Fig. 3, one can observe that the occurrence of more than fault does not create noticeable distortions within the dataset.

The utility of the proposed method will focus on such datasets, and fleets FD001 and FD003 will not be further discussed in this work, as they do not exhibit varying operating conditions. A more detailed description of the FD002 and FD004 datasets is presented in Section 6.

5. The proposed framework

The MLDR-framework minimizes discontinuity by learning a cluster set grouping of observations based on the dataset's modality. This modality is defined by a kernel density estimation (KDE), where the bandwidth of the KDE is parameterized through a heat diffusion solution to the approximate mean integrated squared error. Following an optimized clustering of related observations, the low-dimensional representation of each cluster is learned using Laplacian eigenmaps. Finally, the original time sequence of each observation across the low-dimensional clusters is used to re-index the low-dimensional representations into a continuous low-dimension feature set.

Generally, manifold learning algorithms generate distorted affinity matrices when observations are discontinuous, and therefore have difficulties in learning an accurate low-dimensional embedding [31]. The MLDR-framework's clustering initialization aims to improve the approximation of the data's affinity matrix during the embedding process by reducing the matrix sparsity generated by observations that are similar with respect to index sequence, but differ due to high ranging parameter variations (i.e. discontinuous). In this discussion, we restrict our focus to high-dimensional data with discontinuity due to a set of features with samples generated from a wide range of varying parameters due to transient operating conditions.

The proposed MLDR-framework $\varphi(B, C, t)$, is explained as follows:

$$\varphi(B, C, t): B(t) \rightarrow U(t). \quad (12)$$

Here, B is the original dataset that describes the machine's sensor measurements, and $C = \{c_1, c_2, c_3, \dots, c_n\}$ is the determined cluster set of n number of clusters that provide an optimized grouping of the observations in B . Also, $U(t)$ is the lower-dimension representation, and $t: \{t_1, \dots, t_\nu\}$

denotes the index of observations in the dataset with ν observations.

5.1. MLDR-framework algorithm

Given a high-dimensional discontinuous dataset $B \in \mathbb{R}^{\nu \times m}$, where m is the number of features and ν is the number of observations:

1. The number of modes n in B is determined and used to define the number of clusters generated by transient operating conditions. A Gaussian KDE with a probability density function (pdf) $g(x; \tau)$, is estimated for each feature where

$$g(x; \tau) = \frac{1}{\nu} \sum_{j=1}^{\nu} \phi(x_j, \Xi; \tau), \quad (13)$$

and

$$\phi(x_j, \Xi; \tau) = \frac{1}{\sqrt{2\pi\tau}} e^{-\frac{(x_j - \Xi)^2}{2\tau}}$$

is the Gaussian kernel with location Ξ and bandwidth τ of an observed vector $x: \{x_1, \dots, x_\nu\}$ with ν observations. In this demonstration, x represents a feature $b(t)^i$, where $i = 1, \dots, m$. Assuming a properly estimated τ , the average number of modes identified across all features is then used as the number of modes n to describe the number of discontinuities in the observed dataset, as illustrated in Fig. 6.

The estimation of τ is a nontrivial solution, and common methods like Silverman's rule [41] are conservative in estimation. They do not accurately describe the data's true distribution, and may not accurately describe characteristics such as modality [9].

For a more accurate solution of τ , we explore the measure of Mean Integrated Squared Error (MISE):

$$MISE = E_f \int (\hat{f}(x) - f(x))^2 dx,$$

where $f(x)$ is an unknown density, and $\hat{f}$ is the estimate of the unknown density. In this work, the asymptotic approximation to the Mean Integrated Squared Error, approximate mean integrated squared error (AMISE) for the Gaussian KDE is applied. The minimization of the AMISE, described in Eq. (14), is the optimal bandwidth τ [9]. A full explanation of AMISE is provided in [9].

$$\begin{aligned} \tau &= \underset{\tau > 0}{\operatorname{argmin}} \operatorname{AMISE}\{\hat{f}\}(\tau) \\ &= (2\nu\sqrt{\pi} \|f''\|^2)^{-2/5}. \end{aligned} \quad (14)$$

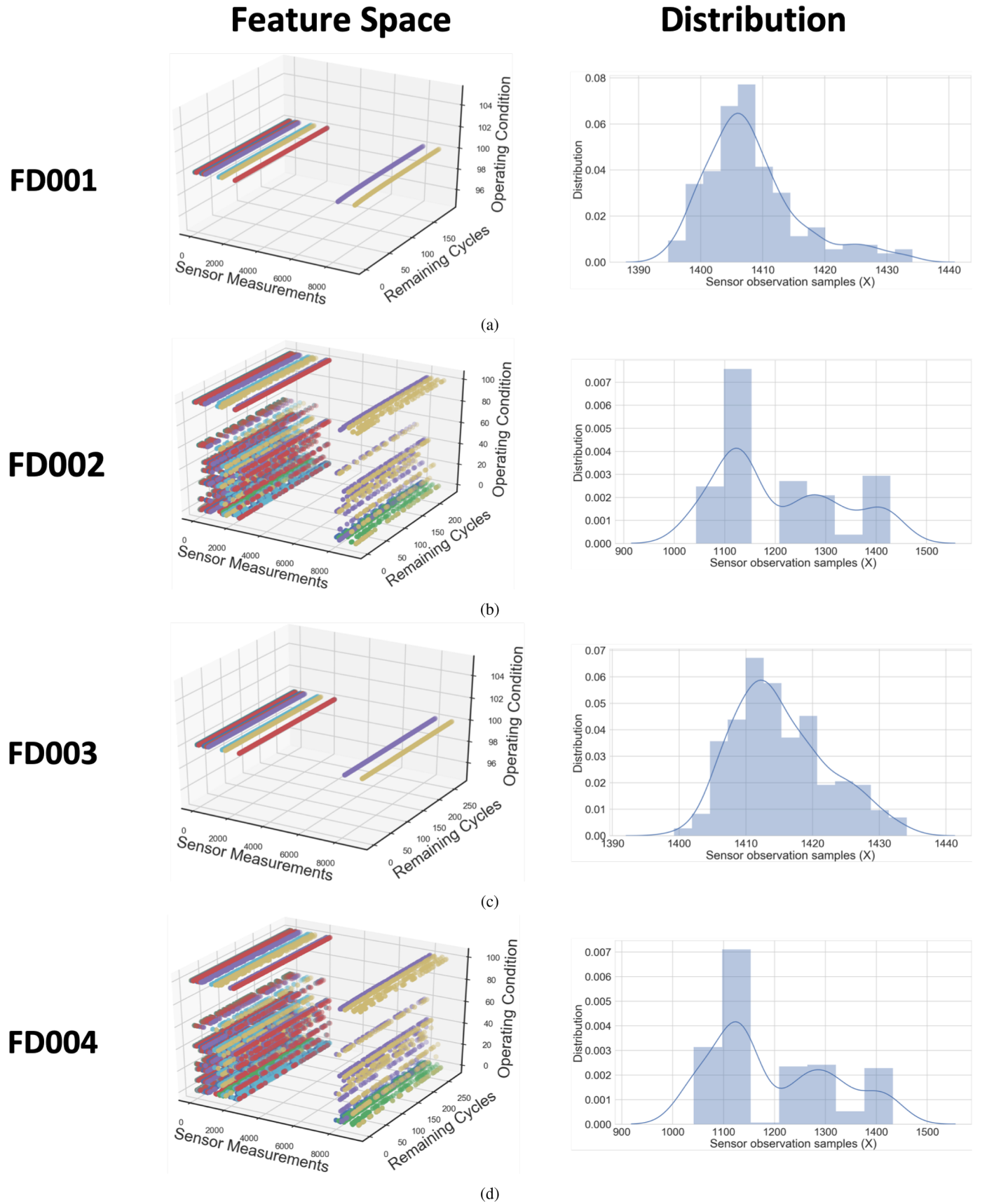
In Eq. (14), ν denotes the number of observations in x , and f'' is the second order derivative of the Gaussian KDE. Although the AMISE provides a plug-in method to estimate the bandwidth, one is required to define the function that describes the Gaussian KDE. This work utilises a solution that explores the relationship between the Gaussian KDE and the heat equation as described in [9]. The Gaussian KDE satisfies the heat diffusion partial differential equation (PDE) [9]:

$$\frac{\partial}{\partial t} f(x) = \alpha \frac{\partial^2}{\partial x^2} f(x), \quad x \in \mathbb{R}^\nu, t > 0, \quad (15)$$

with a diffusion constant α , an initial condition $f(x; 0) = \Delta(x)$. Where $\Delta(x)$ is the empirical density of the observed data $X \in [0, 1]$, and a time index t . The boundary condition of Eq. (15) is

$$\left. \frac{\partial}{\partial x} f(x) \right|_{x=1} = \left. \frac{\partial}{\partial x} f(x) \right|_{x=0} = 0.$$

Although x is in the domain $x \in [0, 1]$, there is no loss of generality, as any bounded domain can be scaled to a $[0, 1]$ range by a linear function [9]. The solution of the diffusion equation provides a Gaussian KDE $g(x; \tau)$ where the time of convergence t denotes the



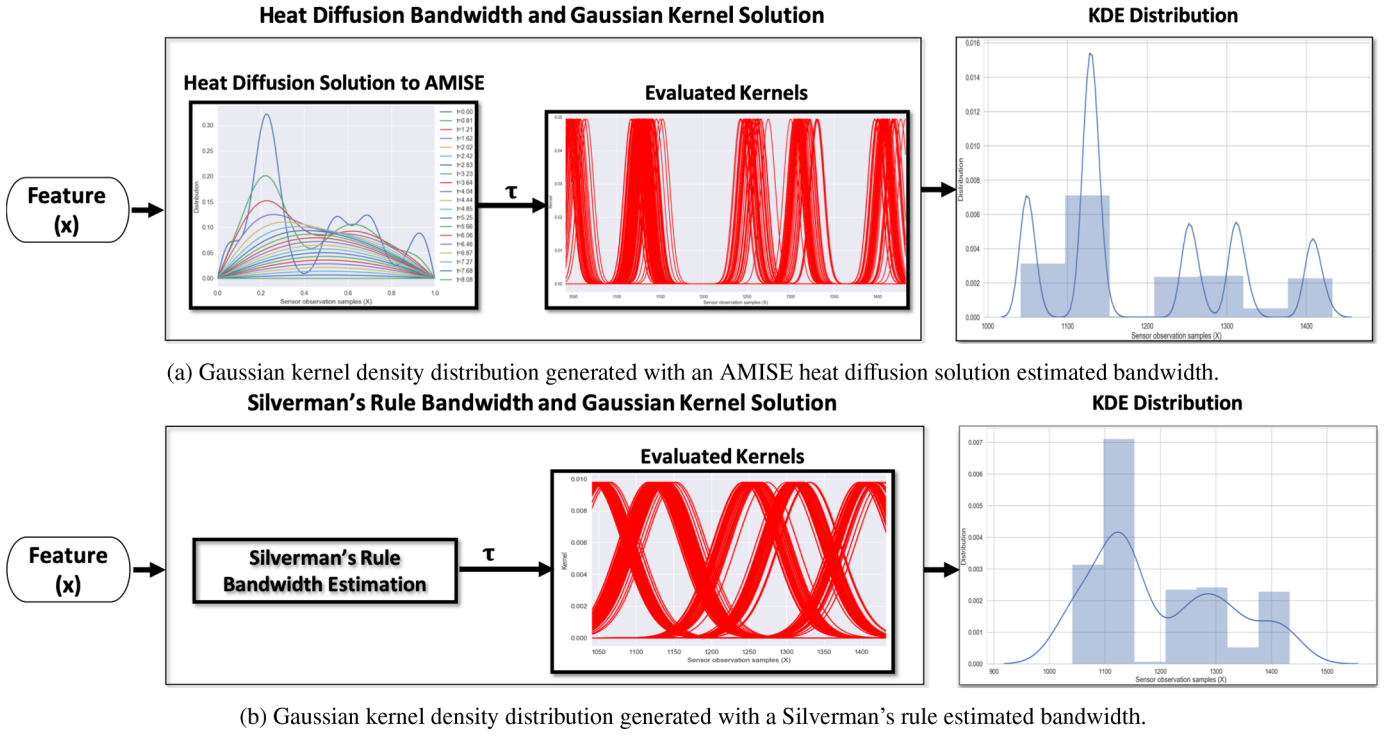


Fig. 4. Comparison between Gaussian kernel density estimated (KDE) distribution of a feature generated with a AMISE heat diffusion solution estimated bandwidth (a), and a Silverman's rule estimated bandwidth (b). The same feature was used in both (a) and (b).

bandwidth τ (i.e. $t = \tau$) [9]. Given the Gaussian KDE is the Green's function for Eq. (15), the solution converges to a uniform density as $t \rightarrow \infty$. Fig. 4 illustrates the difference between a *Silverman's rule* and a diffusion solution to AMISE estimated bandwidth affect on a learned Gaussian KDE distribution of a feature. The interested reader is directed to [9] for a complete proof of the proposed theory. Botev et al. [9] provided the theoretical foundations to the heat diffusion AMISE solution for τ . In this work we provide a unique function for the empirical distribution (Eq. (17)) that allows for an applicable, and computationally feasible implementation of the proposed theory in [9]. This work's solution of the heat diffusion (Eq. (15)) to achieve a paramertized KDE is constructed through the following:

$$\frac{\partial}{\partial t} f(x; \tau) = \frac{1}{2} \frac{\partial^2}{\partial x^2} f(x; \tau), \quad x \in X, t > 0, \quad (16)$$

where the initial condition is $f(x; t = 0) = \Delta(x)$.

We define the empirical distribution as:

$$\Delta(x) = \frac{g(x; \tau^*) - \min(g(x; \tau^*))}{\max(g(x; \tau^*)) - \min(g(x; \tau^*))}, \quad x \in [0, 1], \quad (17)$$

in which τ^* is the Silverman's rule estimation for a distribution's bandwidth [41],

$$\tau^* = 1.06\sigma\nu^{-1/5}.$$

Also, σ is the standard deviation and ν is the number of observation. Silverman's rule is applied as a conservative estimation to give a general description of the data. Note, the formulation of Eq. (17) ensures that the unit probability is the highest, which scales the function to ensure the distribution is integrable. For similar reasons, the function $f(x; t)$ in Eq. (16) is also adjusted by:

$$f(x; t > 0) = \frac{g(x; \tau)}{\nu},$$

where ν is the number of observations. Also, this form of linear scaling allows for the solutions of Eq. (16) to be computationally

feasible in each iteration. Fig. 5 illustrates the solution steps (t) of a diffusion estimated bandwidth for a KDE distribution achieved using the approach described in this step. Also, Fig. 4a demonstrates the Gaussian KDE distribution learning process for one feature. The average number of nodes n observed from all features is used to define the number of clusters required.

- Given the number of modes n exposed from the previous step, samples are then assigned to each cluster to form a cluster set $C: \{c_1, c_2, \dots, c_n\}$ that satisfies the *k-means* objective function,

$$\operatorname{argmin}_C \sum_{i=1}^n \sum_{b \in c_i} \|b - \mu_i\|^2, \quad (18)$$

where μ_i is the mean of points in c_i . The *k-means* optimization learns a cluster set $C \in \mathbb{R}^n$, where the within group variance of each cluster is

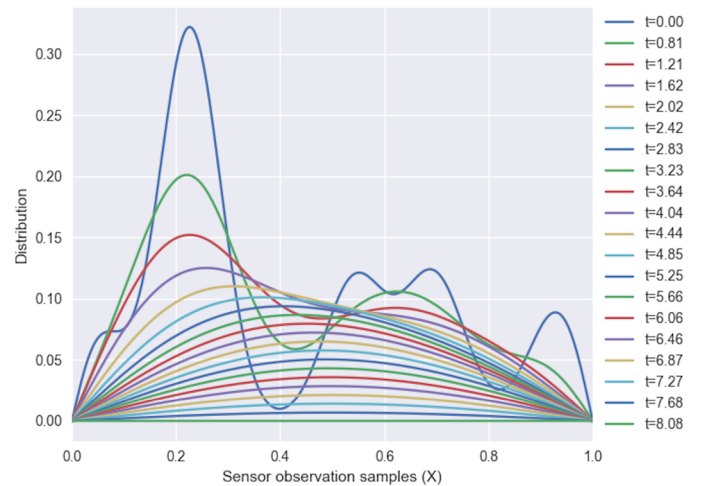


Fig. 5. The solution process of a distribution based diffusion. Time step t at which the diffusion distribution reaches a uniform zero ($t = 8.08$) is the ideal bandwidth τ .

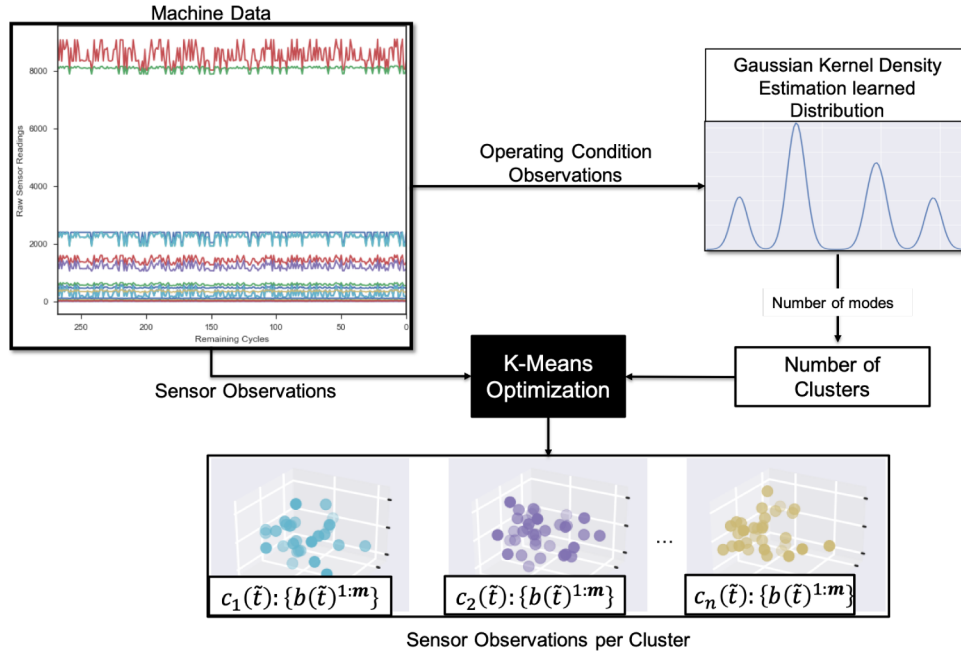


Fig. 6. The MLDR-framework learns the grouping relationship for an cluster set $C: \{c_1: n\}$, with n number of clusters, for a high-dimensional discontinuous machine dataset.

minimized. This process is illustrated in Fig. 6.

For every $c \in R^n$, $\tilde{t}: \{t_{1:\tilde{v}}\}$ denotes the indices of row vectors or samples in a cluster with $\tilde{v}$ samples,

$$c_i: \{b(\tilde{t})^{1:m}\}, 1 < i < n. \quad (19)$$

- Assuming an optimized cluster set C , and per the mathematical definition of a manifold, the group of points in $c_1: n$ can each form a smooth and continuous manifold within each cluster. As described in Fig. 7, the dimension of each cluster is reduced per a mapping function D , for all $c_1: n$:

$$D: c_i \rightarrow u_i \in R^{\tilde{t} \times m^*}, \quad (20)$$

where $U \in R^n$, and $m^* < m$.

The mapping function D embeds the observed points in each c_i into a low-dimensional space u_i . In this presentation of the proposed MLDR-framework, the lower-dimensional space is defined by the Laplacian eigenmap embedding. The Laplacian eigenmap embedding creates a weighted graph $G(W, P)$, of p nodes connected by neighboring points defined by k nearest neighbors.

The number of nodes p is equal to the number of data points observed in the high-dimensional space. The weighting W of the edges or connections for connected nodes are describe by

$$W_{ij} = e^{-\frac{\|p_i - p_j\|^2}{2k^2}}. \quad (21)$$

Otherwise $W_{ij} = 0$, and

$$V_{ii} = \sum_{j \in p} W_{ij}. \quad (22)$$

Here V is a diagonal matrix and k is a free parameter chosen a priori. In this work, k is equal to $\tilde{v}$ number of points within a cluster. There are many methods in which the W_{ij} can be constructed. In this work, the Gaussian kernel (rbf) in the form of Eq. (21) is applied. Using the weighted graph, the embedding map D is obtained by

$$D = \min_Y \text{Tr}(Y^T L Y), \quad (23)$$

where the Laplacian function $L = V - W$. The embedding vector is assumed to be of a unit covariance $Y^T Y = I$. By solving the eigenvalue problem,

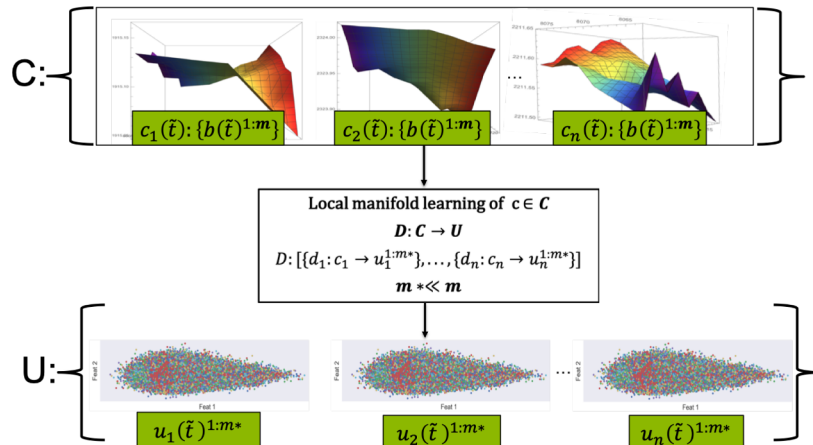


Fig. 7. The high-dimensional manifold of each cluster $c_i \in C$ is mapped to a low-dimensional cluster $u_i \in U$ using a mapping function D .

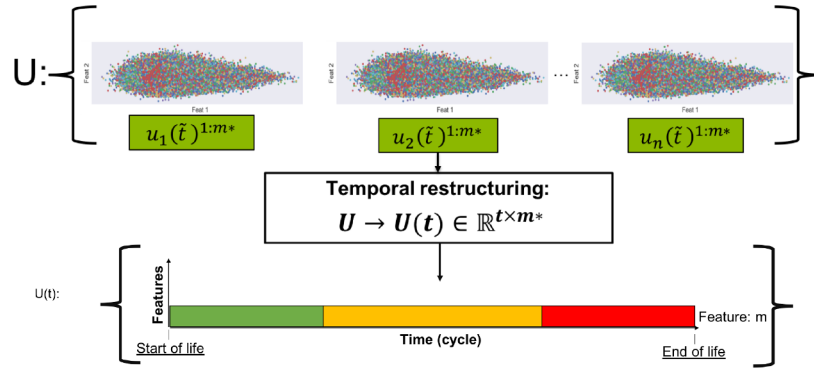


Fig. 8. The original time sequence t of each observation across the low-dimensional clusters $u \in U$ is used to re-index the low-dimensional representations into a continuous low-dimension time series feature set $U(t)$.

$$LY = \lambda VY, \quad (24)$$

the solutions of Y as the eigenvectors of L provide the lower dimension subspace of the original data $Y = u_i \in \mathbb{R}^{i \times m^*}$.

4. As illustrated in Fig. 8, the low-dimensional observations in the cluster set U are re-indexed per the order of observation in the original dataset,

$$t: U \rightarrow U(t) \in \mathbb{R}^{t \times m^*}. \quad (25)$$

This forms the low-dimension time series representation,

$$U(t): \{u(t)^1, u(t)^2, \dots, u(t)^{m^*}\}, \quad (26)$$

where $U(t)$ is the lower-dimension representation of the original data, and $m^* < m$.

The proposed MLDR-framework provides a data-driven method to reduce the dimensionality of a high-dimensional discontinuous dataset. The main contribution of the MLDR-framework is the data-driven method of characterizing the influence of transient operating conditions as discontinuities, through identified number of modes in the observed dataset.

Theoretically, any nonlinear or manifold learning method can be implemented in step 3. In this work, we selected LEM as it is robust to high variations of noise, which allowed for a more reliable dimension reduction. Likewise for step 2, any well established method of partition clustering can be applied to generate the optimal grouping of samples per the identified number of modes.

6. Experiments

In this section we evaluate the performance of the MLDR-framework, and standard LLE, Isomap, and LEM generated representations on the *prognosis of remaining useful life (RUL)*, *classification of normal and failure modes*, and *anomaly detection of fault indicators*. These methods are often implemented using ML algorithms of *regression*, *classification*, and *anomaly detection* respectively. As these are the main components of ML-based PdM algorithms, the following experiments show the utility of the MLDR-framework when using such algorithms on nonlinear discontinuous machine data. Also, the following experiments demonstrate ML algorithms' susceptibility to high dimensional data, and poorly dimension reduced data. The Python scripts used to test this method can be found at the following link: <https://github.com/ooaremu/ML-framework>.

6.1. Datasets

The field of PdM has limited number of available datasets [43]. Also, due to commercial confidentiality, there are limited numbers of publicly available machine datasets. Due to this constraint, the

evaluation of the MLDR-framework in this work is limited to the simulated C-MAPSS FD002 and FD004 datasets, as they exhibit the properties of high-dimensional discontinuous data and describe typical systems use in the domain of PdM.

As described in [39], the C-MAPSS datasets consists of time series data describing the degradation of the different engines in each fleet by using 24 sensors (features) that are contaminated with noise. Of the 24 sensors, three (3) measure varying operating conditions, and 21 sensors measure the respective systems' life cycle behavior. It should be noted that the life cycle length of all engines vary. Also, each engine contains unknown manufacturing variation and varying degrees of initial wear that are considered normal (i.e. non-faulty) [39]. Each engine in the datasets are referenced by their unit numbers.

6.1.1. Dataset 1

Dataset 1, FD002, is a fleet of 260 turbofan engines that have experienced 6 transient operating conditions, and each with varying life cycle length. This fleet experienced a degradation to failure due to a failed high-pressure compressor (HPC).

6.1.2. Dataset 2

Dataset 2, FD004, is a fleet of 248 turbofan engines (different from FD002). This fleet has experienced 6 transient operating conditions in which the engines have experienced a degradation to failure due to a failed HPC and fan.

6.2. Prognosis of remaining useful life

The prognosis of a machine's remaining useful life provides an estimate to the remaining life cycles of a machine prior to maintenance or occurrence of failure. The ability to accurately predictive when a machine will fail is one of the most crucial components of a reliable PdM tool. The accuracy of this component is heavily dependent on the algorithms applied and how they are trained.

As mentioned, the data and the dimensionality of the data used to train such algorithms can heavily influence the performance. In this experiment, the effects of high dimensionality and data discontinuity on the performance of ML algorithms is depicted. Also, the MLDR-framework's ability to circumvent the curse of dimensionality is presented.

A convolutional neural network remaining useful life (CNN-RUL) prognosis algorithm described in Babu et al. [4] is applied to the original datasets, the MLDR-framework, LLE, Isomap, and LEM representations. In this work, the same algorithm with the same parameterization and structure, as explained in Babu et al. [4], is used for the original data. With the generated representations, the CNN is trained and tested with the evaluated dimension reduction process, while the neural network structure and processing sequence is unaltered.

This evaluation demonstrates that reducing the dimensionality of

datasets FD002 and FD004, with the proposed MLDR-framework as a component to the training process of a ML-based prognosis tool, can improve algorithm performance. The metrics used in this experiment to measure the performance of each method are Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) to measure the respective errors between the predicted and actual RUL. Per the measure of each metric, the lower the score, the better the performance of the tested algorithm.

When applying the CNN-RUL prognostics algorithm from Babu et al. [4], the feature and label set-up described was applied. Each engine's lifecycle was segmented into multiple multivariate time series sequence lengths of 15, and associated to a corresponding RUL obtained from the cycle time of the 15th sample point. Therefore, each engine's life cycle is composed of multiple $15 \times m$ or $15 \times m^*$ sequences. Per standard ML algorithm training process, in each fleet dataset, 70% and 30% of the engines are selected for training and testing, respectively.

6.3. Classification of normal and failure modes

As mentioned in Section 2.1, the ability to perform diagnostics analysis in form of differentiating normal conditions from failure modes is critical to successfully applying a PdM tool. In terms of data analysis, this ability corresponds to differentiating data points that correspond to normal operating modes and failure modes. With respect to ML, this is often performed using classification algorithms. Classification algorithms assign an observed variable or set of data points to a predefined class or label [19].

In this experiment we display the effects of high dimensional continuous data on select ML algorithms' ability to perform basic classification of data points corresponding to normal and failure states. Although there exists various classifiers, this work will focus on commonly used methods of: *decision tree* [37], *support vector machine* (SVM) [13] and *kernel support vector machine* (rbf-SVM) [48].

As all engines in both datasets are run-to-failure, it is assumed that the first set of L_{norm} cycles, describe an engine's normal state and the last set of L_{fail} cycles describe its failure/failing state. This allowed for a reliable naive separation of the normal and failure data points within each engine's life cycle. The training dataset comprised of the sampled normal and failure data points. This is further explained in Fig. 9.

In data preparation, an engine's life cycle, X_c , in both dataset were reduced to only consist of L_{norm} and L_{fail} samples and were labeled per respective state. The training set consisted of two labeled classes assumed to describe the normal and failure states of the engines. Each training set $\psi = \{(X, y_i)\}$ where X is the selected samples and y_i is the associate label is generated by

$$\psi = \{(X_i, y_0) | t_i = L_{fail}\} \cup \{(X_i, y_1) | t_i = L_{norm}\}. \quad (27)$$

Here t_i is the life cycle set spanned by the observed data point in the labeled classes.

In each dataset, the first 50 cycles of each engine were labeled as normal data, and the last 50 cycles of each engine are labeled as failure observations. Thus, for FD002 and FD004, the L_{norm} set consists of 50 samples and L_{fail} set, consists of 50 samples. The number of samples in each class ensure that an adequate sample of each engine's states (normal and failure) are well represented. The average life cycle of each engine in FD002 is 325 cycles, while the engines in FD004 have an

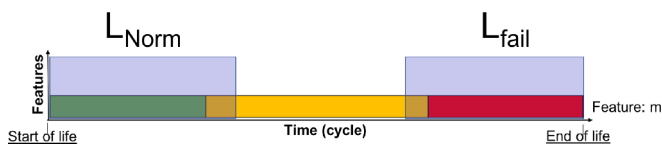


Fig. 9. An example of the selection of L_{norm} and L_{fail} samples in a run-to-failure life cycle with m number of features, in which end of life was observed due to failure.

average life cycle of 450 cycles.

Metrics of average accuracy, average precision, and area under the receiver operating characteristic curve (AUC) are used to measure the performance of the classifiers per learned low-dimensional representation and original data. All scores are normalized to [0:1], where 1 is a perfect score.

6.4. Anomaly detection of fault indicators

In PdM, anomaly detection is the ability to detect unfamiliar operating conditions that vary from known normal operating conditions. Thus, providing the ability to know when a fault has occurred within a machine. Within ML, anomalies are observed data samples that do not contain the same characteristics as the majority of the dataset which are deemed normal samples [28].

Anomaly detection algorithms determine if newly observed data points do not match a predefined learned pattern [8]. The anomaly detectors used for the MLDR-framework utility analysis are *One-Class SVM* (OCSVM) [35] and *Isolation Forest* (iForest) [28].

6.4.1. One-class SVM

OCSVMs are trained with unlabeled observations. Newly observed data is then measured against learned data using a distance measure to determine level of similarity. Observations that strongly deviate from learned data are identified as outliers.

6.4.2. Isolation forest

The iForest algorithm is also trained with unlabeled observations, but its detection of anomaly is not based on a distance measure from normal data. This method operates on the basis that anomalies are rarely observed, and are more susceptible to isolation [28].

For anomaly detection performance evaluation, the training set $\psi = \{(X)\}$ consists of unlabelled data points that describe an engine's normal state i.e the learned normal data

$$\psi = \{(X_i) | t_i = L_{norm}\}. \quad (28)$$

The anomaly detector learns the normal state of the engine from the training sample points. In each experiment, the anomaly detection algorithm is exposed to a test set which contains known normal and failure data points X_{ad} .

$$X_{ad} = \{(X_i) | t_i = L_{fail}\} \cup \{(X_i) | t_i = L_{norm}\}. \quad (29)$$

The metrics of false positive (FP) (failure observations are incorrectly interpreted as normal), true positive (TP) (correctly detected normal observations), false negative (FN) (normal observations are incorrectly interpreted as failure), true negative (TN) (correctly detected failure observations) were used. The accuracy of each detector per representation is also measured.

7. Results

In this section, we review the performance of the MLDR-framework on both datasets. Also, we measure the utility of the MLDR-framework representation against the original data based on algorithm performance in the categories of *regression*, *classification*, and *anomaly detection*. Low-dimensional representation from other manifold learning methods are also included for comparison. The results are the direct output of the dimension reduction methods, as no further post-processing or other methods of data manipulation was performed. In this work, all generated representation are presented in a 2D vector space (i.e. $m^*=2$). This is to achieve the lowest possible dimension while ensuring applicability to ML algorithms, as ML algorithms generally require more than one feature for statically relevant inference [37].

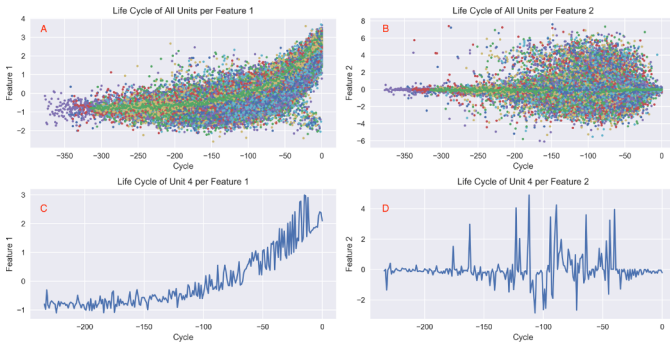


Fig. 10. Machine learning dimension reduction framework (MLDR-framework) applied to FD002 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

7.1. Dimension reduction of dataset 1 (FD002)

Figs. 10–13 show the MLDR-framework, Isomap, LLE, and LEM's low-dimensional representation of the original data respectively. Visually, the MLDR-framework outperforms the other manifold learning methods. As shown in Fig. 10, the MLDR-framework is able to generate features that provide a trend model that shows a clear degradation pattern of each engine's complete run-to-failure life cycle. In plot D of Fig. 10, one can observe a signal approximately between -175 and -150 cycles that signifies a deviation from the engine's steady state.

This assumption is further supported by plot C of the same figure. During this deviation approximately between -175 and -150 cycles, plot D shows a strong signal of greater value than the samples in prior cycles. This strong signal in plot D of Fig. 10, could indicate the cycle in which a fault occurred during the engines life cycle.

Isomap, LLE, LEM do not show any decipherable trend that is as noticeable as in the MLDR-framework's representation. This is an indicator that the MLDR-framework representation is visually more useful than the original data and the representation of the mentioned manifold learning methods. This highlights the need for the MLDR-framework when observing multivariate time series machine data in which the machine has undergone multiple operating conditions.

7.2. Dimension reduction of dataset 2 (FD004)

Given FD004, a different set of jet engines that experienced different operating conditions, the MLDR-framework is also able to generate a set of lower-dimension features that are more interpretable than the original data and other representations from the mentioned manifold learning methods. Figs. 14–17 show the MLDR-framework, Isomap,

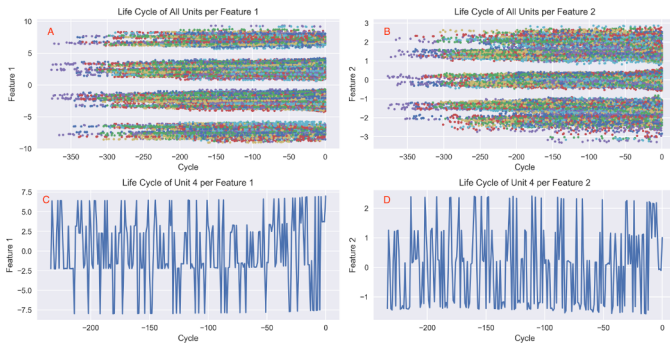


Fig. 11. Isometric Feature Map (Isomap) applied to FD002 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

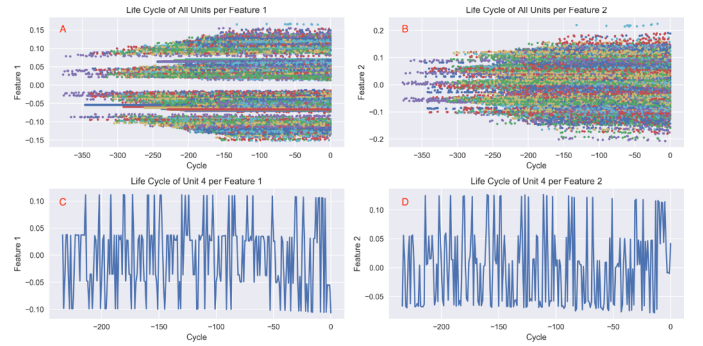


Fig. 12. Locally Linear Embedding (LLE) applied to FD002 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

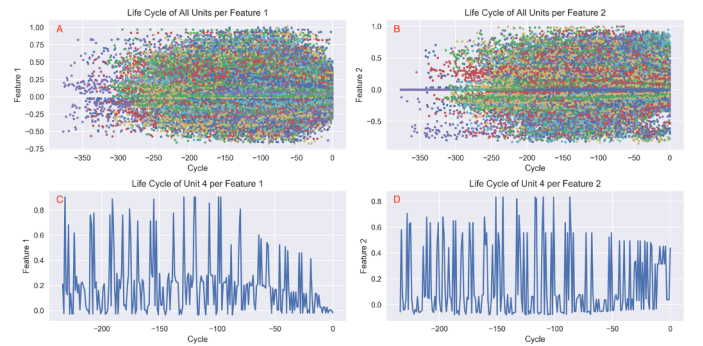


Fig. 13. Laplacian Eigenmaps (LEM) applied to FD002 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

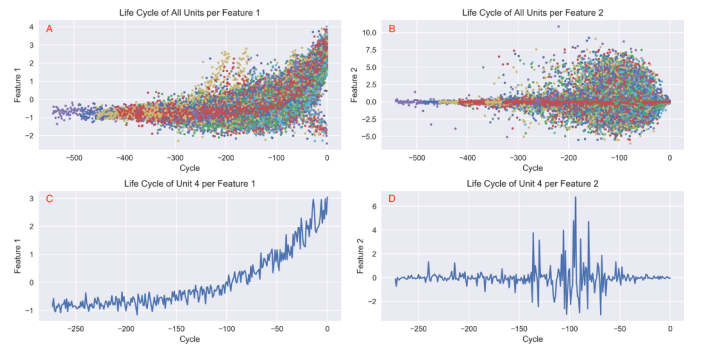


Fig. 14. Machine learning dimension reduction framework (MLDR-framework) applied to FD004 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

LLE, and LEM's ability to reduce the feature to a 2 feature representation, respectively.

Again, in plot D of Fig. 14, one can identify a signal that indicates a deviation from the engine's steady state observed in plot C of Fig. 14. This observation, approximately between -230 and -150 cycle, could be a signal that indicates the occurrence of a fault. Also, Isomap, LLE, LEM did not show any improvement in providing a useful low-dimension representation when applied to FD004.

7.3. Prediction results

Results of the *prognosis of remaining useful life* experiment provide

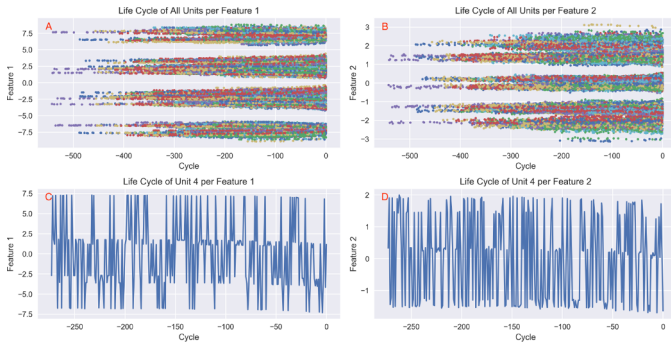


Fig. 15. Isometric Feature Map (Isomap) applied to FD004 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

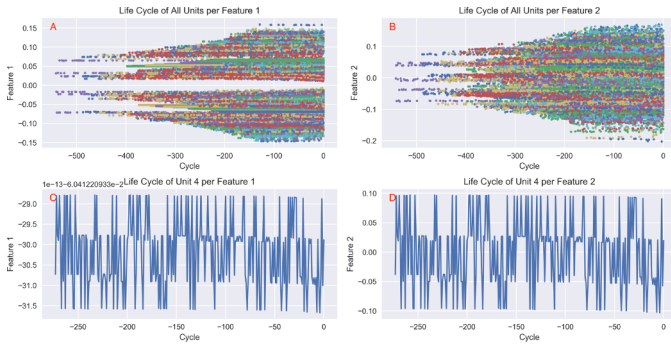


Fig. 16. Locally Linear Embedding (LLE) applied to FD004 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

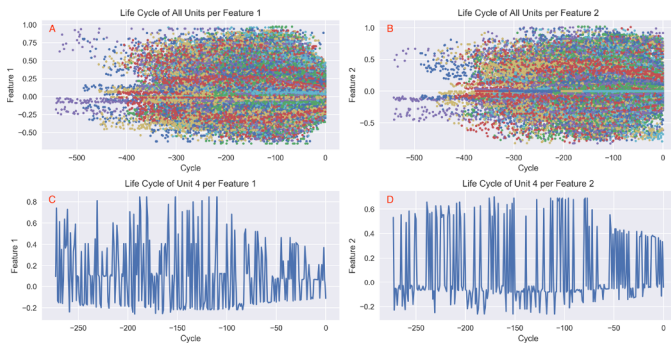


Fig. 17. Laplacian Eigenmaps (LEM) applied to FD004 to achieve a 2 feature dimension representation. The first row is the Feature 1 (A) and Feature 2 (B) time series representation of all units. The second row shows the Feature 1 (C) and Feature 2 (D) time series representation of engine unit 4.

Table 1
CNN-RUL [4] Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) prediction performance on feature projection representations of **FD002**.

Reduction method	Number of Features	RMSE	MAE
Original Data	24	68.91	56.87
MLDR-framework	2	35.27	25.10
Isomap	2	70.10	57.63
LLE	2	68.42	56.30
LEM	2	65.94	54.01

evidence that the application of raw machine data to the DNN-based prognostics tool in [4], limited its performance capability. Furthermore, applying generic dimension reduction methods that only manage

Table 2
CNN-RUL [4] Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) prediction performance on feature projection representations of **FD004**.

Reduction method	Number of features	RMSE	MAE
Original Data	24	90.66	73.59
MLDR-framework	2	51.58	36.78
Isomap	2	90.19	72.78
LLE	2	90.52	73.21
LEM	2	83.67	66.35

Table 3
Classification results per reduction method applied to FD002.

Reduction method	Classifier	Number of features	Average accuracy	Average precision	AUC
Original Data	SVM	24	0.54	0.5	0.63
Original Data	Decision Tree	24	0.73	0.7	0.78
Original Data	RBF SVM	24	0.68	0.6	0.65
MLDR-framework	SVM	2	0.71	0.93	0.97
MLDR-framework	Decision Tree	2	0.95	0.96	0.98
MLDR-framework	RBF SVM	2	0.95	0.96	0.98
Isomap	SVM	2	0.49	0.52	0.48
Isomap	Decision Tree	2	0.7	0.67	0.77
Isomap	RBF SVM	2	0.6	0.6	0.52
LLE	SVM	2	0.49	0.51	0.49
LLE	Decision Tree	2	0.65	0.62	0.69
LLE	RBF SVM	2	0.6	0.6	0.52
LEM	SVM	2	0.44	0.44	0.6
LEM	Decision Tree	2	0.68	0.67	0.68
LEM	RBF SVM	2	0.6	0.6	0.6

Table 4
Classification results per reduction method applied to FD004.

Reduction method	Classifier	Number of features	Average accuracy	Average precision	AUC
Original Data	SVM	24	0.53	0.55	0.71
Original Data	Decision Tree	24	0.81	0.77	0.84
Original Data	RBF SVM	24	0.59	0.59	0.66
MLDR-framework	SVM	2	0.96	0.96	0.97
MLDR-framework	Decision Tree	2	0.98	0.98	0.99
MLDR-framework	RBF SVM	2	0.96	0.97	0.98
Isomap	SVM	2	0.53	0.49	0.5
Isomap	Decision Tree	2	0.82	0.8	0.84
Isomap	RBF SVM	2	0.59	0.59	0.51
LLE	SVM	2	0.55	0.51	0.48
LLE	Decision Tree	2	0.65	0.64	0.69
LLE	RBF SVM	2	0.59	0.59	0.52
LEM	SVM	2	0.48	0.48	0.47
LEM	Decision Tree	2	0.6	0.59	0.72
LEM	RBF SVM	2	0.59	0.59	0.55

nonlinearity is not enough to improve ML algorithm performance. These results are further detailed in Table 1 for FD002 and Table 2 for FD004.

The application of generic manifold learning algorithms does not guarantee a discernible retention of information contained in the original data that could be used to predict an engine's RUL. By applying the MLDR-framework to generate a low-dimensional representation, while minimizing data discontinuity, one is able to guaranty a more reliable ML and prognostics algorithm performance without losing the original information contained in the original data.

In both datasets, the MLDR-framework's representation proved to command a more reliable and accurate algorithm performance for all applied algorithms. To further emphasize the hindrance of high-dimensional data on ML algorithms, the MLDR-framework applied algorithms achieved better performance while using only 2 features. With respect to addressing discontinuity, the MLDR-framework also provided better algorithm performance compared to generic manifold learning methods

Table 5

Anomaly detection results per reduction method applied to FD002. Detector accuracy, true positive (TP), true negative (TN), false positive (FP), and false negative (FN) are used to evaluate each detector and reduction method combination.

Reduction method	Anomaly detector	Accuracy (%)	Detected normal (TP) (%)	Detected failure (TN) (%)	Undetected failure (FP) (%)	Undetected normal (FN) (%)
Original Data	OCSVM	61.85	60.3	65.1	34.9	39.7
Original Data	iForest	93.99	95.3	39.3	60.7	4.7
MLDR-framework	OCSVM	94.25	92.6	97.7	2.3	7.4
MLDR-framework	iForest	93.99	92.3	96.4	3.7	7.7
Isomap	OCSVM	27.73	29.3	71.5	28.5	70.7
Isomap	iForest	93.99	89.4	43.4	56.6	10.6
LLE	OCSVM	42.56	47.7	54.2	45.8	52.3
LLE	iForest	93.99	91.2	27.1	72.9	8.8
LEM	OCSVM	74.38	69.0	24.1	75.9	31.0
LEM	iForest	93.99	91.2	6.3	93.7	8.8

Table 6

Anomaly detection results per reduction method applied to FD004. Detector accuracy, true positive (TP), true negative (TN), false positive (FP), and false negative (FN) are used to evaluate each detector and reduction method combination.

Reduction method	Anomaly detector	Accuracy (%)	Detected normal (TP) (%)	Detected failure (TN) (%)	Undetected failure (FP) (%)	Undetected normal (FN) (%)
Original Data	OCSVM	56.6	58.6	70.4	29.7	41.4
Original Data	iForest	95.0	91.0	60.4	39.6	9.0
MLDR-framework	OCSVM	91.0	92.8	98.4	1.6	7.2
MLDR-framework	iForest	95.0	95.1	98.0	2.0	4.9
Isomap	OCSVM	45.4	44.0	54.6	45.4	56.0
Isomap	iForest	95.0	90.3	64.5	35.5	9.7
LLE	OCSVM	70.0	70.1	36.6	63.4	29.9
LLE	iForest	95.0	90.7	24.5	75.5	9.3
LEM	OCSVM	40.7	36.5	63.2	36.8	63.5
LEM	iForest	95.0	92.8	3.0	97.0	7.2

which do not have the ability to address discontinuity.

7.4. Classification evaluation results

Per classifier, the MLDR-framework's representation outperformed other comparable manifold learning methods in all metrics of evaluation. Table 3 provides the results of the classification analysis for FD002. The MLDR-framework produced better classification capability than other low-dimension representations, and the original data. Results for FD004 are shown in Table 4. The MLDR-framework also outperforms other reduction methods and the original data in FD004.

7.5. Anomaly detection evaluation results

In anomaly detection performance evaluation, the MLDR-framework outperforms other manifold learning methods in all metrics of evaluation. Table 5 provides the results of the anomaly detection of FD002 observations. The MLDR-framework's usability in basic semi-supervised ML methods is demonstrated by the performance of the anomaly detection algorithms. While the MLDR-framework allow for high anomaly detection rates, it also produced strong rejection of false negatives (FN) and false positives (FP). Results for FD004 are shown in Table 6. In FD004, the MLDR-framework also outperforms other reduction methods.

8. Conclusion

This work provided evidence that machine data should not be treated as general data. In the PdM domain, relying on general analytical methods for machine data analysis can lead to inaccurate predictions or results, and lead to unexpected machine failure and/or economic loss.

A dimension reduction framework was proposed that was designed to circumvent the challenges of high-dimensional discontinuous data, a common characteristic of machine data. The MLDR-framework reduced

the effects of discontinuity in data by defining a cluster set based on the dataset's modality defined by a kernel density estimation. Next, Laplacian eigenmaps are applied to learn the low-dimensional representations of each cluster. Finally, observations across all clusters are sequentially re-indexed to form one continuous low-dimensional feature set. Therefore, providing a low-dimensional representation of the original data that is ideal for ML-driven predictive maintenance.

The MLDR-frameworks capabilities are highlighted in its ability to generate a low-dimension time series representation of discontinuous machine data. The MLDR-framework is able to provide a low-dimensional representation that is useful for immediate predictive analysis. The MLDR-framework representations produced improved ML algorithm performance compared to the original data, and other low-dimensional representations of locally linear embedding, isometric mapping, and Laplacian eigenmaps. The utility of the MLDR-framework was demonstrated on a PdM case study using simulated turbofan jet engine datasets.

The MLDR-framework requires an initialization that evaluates the statistical data per high-dimensional feature. This can generate a computation time bottleneck in very high-dimensional datasets. A method that can conduct the same form of analysis on all high-dimensional features simultaneously would be an improvement. The MLDR-framework should also be tested on additional data sets.

Fault identification and fault type classification, like prognostics, are also important in PdM [2,18,45]. Future research to classify faults per results of the MLDR-framework should be conducted.

Acknowledgment

The authors would like to thank the Australian Government Research Training Program Scholarship, and the American Australian Association Education Fund for their support. Also, the authors acknowledge Amber Elise Koch's contributions in creating the figures for this work.

References

- [1] Amihai I, Chioua M, Gitzel R, Kotriwala AM, Pareschi D, Sosale G, et al. Modeling machine health using gated recurrent units with entity embeddings and k-means clustering. *Proceedings of the IEEE 16th international conference on industrial informatics (INDIN)*. IEEE; 2018. p. 212–7.
- [2] An D, Choi JH, Kim NH. Prognostics 101: a tutorial for particle filter-based prognostics algorithm using matlab. *Reliab Eng Syst Safety* 2013;115:161–9.
- [3] Aremu OO, Palau AS, Parlikad AK, Hyland-Wood D, McAree PR. “Structuring data for intelligent predictive maintenance in asset management”. *IFAC-PapersOnLine* 2018;51(11):514–9. <https://doi.org/10.1016/j.ifacol.2018.08.370>. 16th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2018
- [4] Babu GS, Zhao P, Li X-L. Deep convolutional neural network based regression approach for estimation of remaining useful life. *Proceedings of the international conference on database systems for advanced applications*. Springer; 2016. p. 214–28.
- [5] Bektas O, Alfidail A, Jones JA. Reducing dimensionality of multi-regime data for failure prognostics. *J Fail Anal Prev* 2017;17(6):1268–75.
- [6] Belkin M, Niyogi P. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Comput* 2003;15(6):1373–96.
- [7] Blum AL, Langley P. Selection of relevant features and examples in machine learning. *Artif Intell* 1997;97(1–2):245–71.
- [8] Bonissone P.P., Iyer N.. Soft computing applications to prognostics and health management (PHM): leveraging field data and domain knowledge. *Proceedings of the 9th international work conference on artificial neural networks computational and ambient intelligence, IWANN 2007*;928–939.
- [9] Botev ZI, Grostowski JF, Kroese DP, et al. Kernel density estimation via diffusion. *Ann Stat* 2010;38(5):2916–57.
- [10] Bzdok D, Krzywinski M, Altman N. Machine learning: supervised methods. *Nat Methods* 2018;15(1):5.
- [11] Cai J, Luo J, Wang S, Yang S. Feature selection in machine learning: a new perspective. *Neurocomputing* 2018;300:70–9.
- [12] Chen X-W, Lin X. Big data deep learning: challenges and perspectives. *IEEE Access* 2014;2:514–25.
- [13] Cortes C, Vapnik V. Support vector machine. *Mach. Learn.* 1995;20(3):273–97.
- [14] Do Carmo MP, Fischer G, Pinkall 'U, Reckziegel H. *Differential geometry. Mathematical models*. Springer; 2017. p. 155–80.
- [15] Ellefsen AL, Bjørlykhaug E, Aesøy V, Ushakov S, Zhang H. Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture. *Reliab Eng Syst Saf* 2019;183:240–51.
- [16] Guyon I, Elisseeff A. An introduction to variable and feature selection. *J Mach Learn Res (JMLR)* 2003;3(3):1157–82. 1111.6189v1
- [17] Henderson P, Islam R, Bachman P, Pineau J, Precup D, Meger D. Deep reinforcement learning that matters. *Proceedings of the Thirty-Second AAAI conference on artificial intelligence*. 2018.
- [18] Hong S, Zhou Z, Lu C, Wang B, Zhao T. Bearing remaining life prediction using gaussian process regression with composite kernel functions. *J. Vibroeng.* 2015;17(2).
- [19] Jia X, Jin C, Buzza M, Di Y, Siegel D, Lee J. A deviation based assessment methodology for multiple machine health patterns classification and fault detection. *Mech Syst Signal Process* 2018;99:244–61.
- [20] Kannan R, Manohar SS, Kumaran MS. Nominal features-based class specific learning model for fault diagnosis in industrial applications. *Comput Ind Eng* 2018;116:163–77.
- [21] Khan S, Yairi T. A review on the application of deep learning in system health management. *Mech Syst Signal Process* 2018;107:241–65.
- [22] Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. *Proceedings of the Advances in neural information processing systems*. 2012. p. 1097–105.
- [23] LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature* 2015;521(7553):436.
- [24] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: a systematic review from data acquisition to RUL prediction. *Mech Syst Signal Process* 2018;104:799–834.
- [25] Li X, Ding Q, Sun J-Q. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety* 2018;172:1–11.
- [26] Li X, Zhang W, Ding Q. Deep learning-based remaining useful life estimation of bearings using multi-scale feature extraction. *Reliab Eng Syst Saf* 2019;182:208–18.
- [27] Lin T, Zha H. Riemannian manifold learning. *IEEE Trans Pattern Anal Mach Intell* 2008;30(5):796–809.
- [28] Liu FT, Ting KM, Zhou Z-H. Isolation-based anomaly detection. *ACM Trans Knowl Discov sData (TKDD)* 2012;6(1):3.
- [29] Liu J, An Y, Dou R, Ji H, Liu Y. Helical fault diagnosis model based on data-driven incremental merge. *Comput Ind Eng* 2018;125:517–32.
- [30] Liu L, Wang S, Liu D, Zhang Y, Peng Y. Entropy-based sensor selection for condition monitoring and prognostics of aircraft engine. *Microelectron Reliab* 2015;55(9–10):2092–6.
- [31] Liu Z, Wang J, Liu G, Pu J. Sparse low-rank preserving projection for dimensionality reduction. *IEEE Access* 2019;7:22941–51.
- [32] Najafabadi MM, Villanustre F, Khoshgoftaar TM, Seliya N, Wald R, Muharemagic E. Deep learning applications and challenges in big data analytics. *J Big Data* 2015;2(1):1.
- [33] Ng AY. Feature selection, l1 vs. l2 regularization, and rotational invariance. *Proceedings of the twenty-first international conference on machine learning*. ACM; 2004. p. 78.
- [34] Pai G, Talmon R, Bronstein A, Kimmel R. Dimal: Deep isometric manifold learning using sparse geodesic sampling. *Proceedings of the IEEE winter conference on applications of computer vision (WACV)*. IEEE; 2019. p. 819–28.
- [35] Perdisci R, Gu G, Lee W. Using an ensemble of one-class svm classifiers to harden payload-based anomaly detection systems. *Proceedings of the ICDM'06. sixth international conference on data mining*. IEEE; 2006. p. 488–98.
- [36] Roweis ST, Saul LK. Nonlinear dimensionality reduction by locally linear embedding. *Science* 2000;290(5500):2323–6.
- [37] Russell SJ, Norvig P. *Artificial intelligence: a modern approach*. Malays. Pearson Edu. Limit.; 2016.
- [38] Saul L.K., Roweis S.T.. An introduction to locally linear embedding. unpublished Available at: <http://www.cs.toronto.edu/~roweis/lle/publicationshtml2000>.
- [39] Saxena A, Goebel K. Turbofan engine degradation simulation data set. NASA Ames Prognostics Data Repository 2008.
- [40] Schmidhuber J. Deep learning in neural networks: an overview. *Neural Netw* 2015;61:85–117.
- [41] Scott DW. Scott's rule. *Wiley Interdiscip Rev Comput Stat* 2010;2(4):497–502.
- [42] Shiue Y-R, Lee K-C, Su C-T. Real-time scheduling for a smart factory using a reinforcement learning approach. *Comput Ind Eng* 2018;125:604–14.
- [43] Sikorska J, Hodkiewicz M, De Cruz A, Astfalck L, Keating A. A collaborative data library for testing prognostic models. *Proceedings of the 3rd European Conference Prognostics Health Management Society, Bilbao, Spain*. 2016.
- [44] Skiena S. Dijkstra's algorithm. *Implementing Discrete Mathematics: Combinatorics and Graph Theory with Mathematica*. Reading, MA: Addison-Wesley 1990:225–7.
- [45] Susto GA, Schirru A, Pampuri S, McLoone S, Beghi A. Machine learning for predictive maintenance: a multiple classifier approach. *IEEE Trans Ind Inf* 2015;11(3):812–20.
- [46] Trappey AJ, Trappey CV, Ma L, Chang JC. Intelligent engineering asset management system for power transformer maintenance decision supports under various operating conditions. *Comput Ind Eng* 2015;84:3–11.
- [47] Tsai F. Comparative study of dimensionality reduction techniques for data visualization. *J Artif Intell* 2010;3(3):119–34.
- [48] Widodo A, Yang B-S, Han T. Combination of independent component analysis and support vector machines for intelligent faults diagnosis of induction motors. *Expert Syst Appl* 2007;32(2):299–312.
- [49] Xing X, Du S, Wang K. Robust hessian locally linear embedding techniques for high-dimensional data. *Algorithms* 2016;9(2):36.
- [50] Zambal S, Eitzinger C, Clarke M, Klintworth J, Mechin P-Y. A digital twin for composite parts manufacturing: Effects of defects analysis based on manufacturing data. *Proceedings of the IEEE 16th international conference on industrial informatics (INDIN)*. IEEE; 2018. p. 803–8.
- [51] Zhang B, Zhang L, Xu J. Degradation feature selection for remaining useful life prediction of rolling element bearings. *Qual Reliab Eng Int* 2016;32(2):547–54.